Core AI Engineering

A Complete Guide

10 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

https://snehaliteng.github.io/

Copyright & Disclaimer

> **Please do not print this document unnecessarily.** Think of the environment — save paper and trees. Share the digital link instead: https://snehaliteng.github.io/tutorials/ai-engineer-core/

← Back to Tutorials

Build Your First LLM Product

Running LLMs Locally with Ollama

Ollama lets you run open-source LLMs locally. Install it, pull a model, and make your first API call.

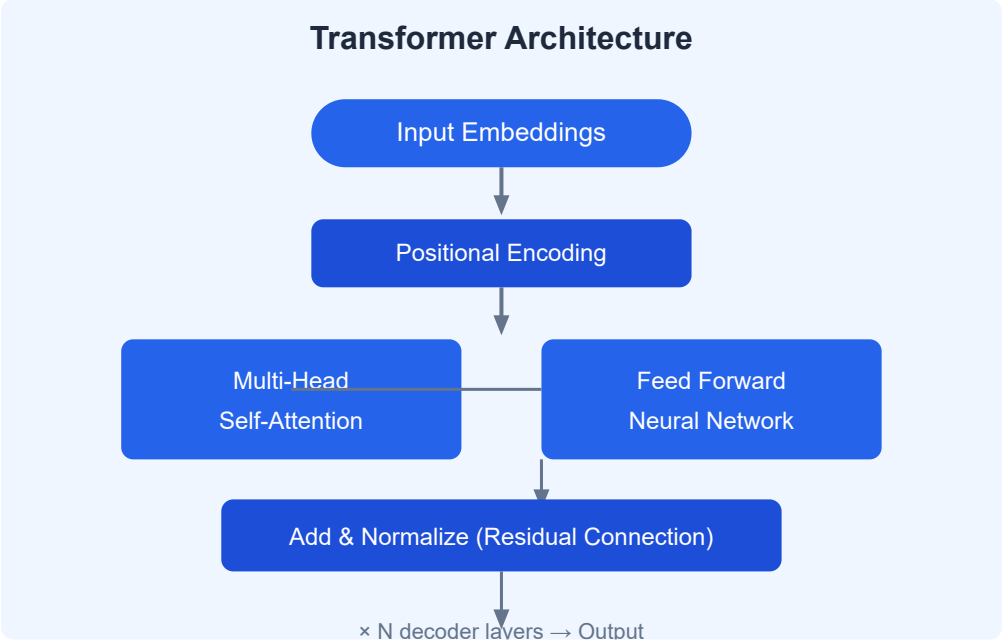
```
# Install Ollama (macOS/Linux)
curl -fsSL https://ollama.com/install.sh | sh

# Pull a model
ollama pull llama3.2

# Python API call
import ollama
response = ollama.chat(model="llama3.2", messages=[
    {"role": "user", "content": "Summarize a website about AI."}
])
print(response["message"]["content"])
```

LLM Engineering Building Blocks

Block	Description
Models	Base, Chat, and Reasoning variants
Tools	Function calling, code execution, search
Techniques	Prompt engineering, RAG, fine-tuning

Understanding LLM Types

- **Base Models** — Predict next token, need instruction tuning
- **Chat Models** — Fine-tuned for conversation
- **Reasoning Models** — Chain-of-thought, math, logic (e.g., o1, DeepSeek-R1)



Transformers Architecture

From LSTMs to Transformers — the key innovation is self-attention, which lets the model weigh the importance of all tokens in the input simultaneously.

```
# Key parameters
# - Parameters: 7B, 13B, 70B (more = more capability)
# - Context Window: 4K, 8K, 128K tokens
# - Tokenization: subword encoding (tiktoken, SentencePiece)
```

Business Applications

Build a sales brochure generator, chain multiple LLM calls, and stream results for real-time UX.

```
# Streaming example
for chunk in ollama.chat(model="llama3.2", messages=[...], stream=True):
    print(chunk["message"]["content"], end="", flush=True)
```

> 🖍️ **Exercise:** Install Ollama, pull the Llama 3.2 model, and build a website summarizer. Given a URL, fetch the page content and use the LLM to generate a 3-sentence summary. Stream the response token by token.

← Back to Tutorials

Build a Multi-Modal Chatbot

Multi-Model Conversations

Use LangChain or LiteLLM to switch between frontier models (GPT-4o, Claude, Gemini) with the same interface.

```
pip install langchain langchain-openai litellm
```

Gradio UIs

Gradio lets you prototype AI interfaces in minutes.

```
import gradio as gr

def chat(message, history):
    return f"You said: {message}"

gr.ChatInterface(chat, title="AI Chatbot").launch()
```

System Prompts & Multi-Shot Prompting

```
system_prompt = "You are a helpful airline assistant."
messages = [
    {"role": "system", "content": system_prompt},
    {"role": "user", "content": "Book a flight from NYC to London."}
]
```

Tool Calling with Airline Assistant

Build an agentic workflow where the LLM calls tools to query an SQLite database for flights.

```
import sqlite3

def search_flights(origin, destination, date):
    conn = sqlite3.connect("flights.db")
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM flights WHERE origin=? AND destination=?",
                   (origin, destination))
    return cursor.fetchall()
```

Multi-Modal Capabilities

- **DALL-E 3** — Generate images from text descriptions
- **Text-to-Speech** — Convert responses to spoken audio
- **Gradio Blocks** — Build complex UIs with tabs, rows, and columns

> 🏏 **Exercise:** Build a multi-modal airline assistant chatbot with: (1) a Gradio ChatInterface, (2) SQLite tool for flight searches, (3) DALL-E integration for destination images, and (4) TTS for spoken responses. Use system prompts to guide the conversation.

← Back to Tutorials

Open-Source Gen AI with HuggingFace

The HuggingFace Platform

HuggingFace hosts 500K+ models, 100K+ datasets, and Spaces for demo deployment. Use Colab with free GPUs for inference.

```
pip install transformers torch diffusers accelerate
```

HuggingFace Pipelines

Quick inference for common tasks without boilerplate.

```
from transformers import pipeline

# Sentiment
classifier = pipeline("sentiment-analysis")
result = classifier("I love AI agents!")
print(result)

# NER
ner = pipeline("ner")
print(ner("John works at Microsoft in Redmond."))

# Question Answering
qa = pipeline("question-answering")
result = qa(question="What is AI?", context="AI is the simulation of human in
print(result)

# Image generation
from diffusers import StableDiffusionPipeline
pipe = StableDiffusionPipeline.from_pretrained("runwayml/stable-diffusion-v1-
image = pipe("A robot writing code").images[0]
```

Tokenizers

Tokenizers convert text to token IDs and back. Different models use different tokenizers.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
tokens = tokenizer("Hello, how are you?", return_tensors="pt")
print(tokenizer.decode(tokens["input_ids"][0]))
```

Transformers & Quantization

Use 8-bit or 4-bit quantization to run large models on limited hardware.

```
from transformers import BitsAndBytesConfig

quant_config = BitsAndBytesConfig(load_in_4bit=True)
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b-chat-hf",
    quantization_config=quant_config
)
```

Applications

- **Token-by-token visualization** — See how models generate text
- **Meeting Minutes with Whisper** — Transcribe audio to text
- **Synthetic Data Generation** — Use LLMs to create training datasets

> 📝 **Exercise:** Use HuggingFace pipelines to build a meeting assistant: (1) transcribe an audio file with Whisper, (2) run NER to extract names and dates, (3) generate a summary with a text generation model, and (4) visualize the token-by-token generation process.

← Back to Tutorials

LLM Showdown

Model Selection Strategy

Scaling laws show that larger models (more parameters + more data) perform better, but diminishing returns apply. Use benchmarks and leaderboards to compare.

Model Size	Use Case	Cost
1-3B	Edge devices, classification	Free (local)
7-13B	Chat, summarization, RAG	Low
70B+	Complex reasoning, code gen	Medium-High
Frontier (GPT-4o, Claude)	Enterprise, multimodal	High

Commercial Use Cases

- **Automation** — Replace repetitive human tasks
- **Augmentation** — Assist humans with analysis, writing, coding
- **Agentic AI** — Autonomous decision-making systems

Code Generation Showdown

```
# Test: Convert Python to C++
prompt = "Convert this Python function to C++: def add(a, b): return a + b"

# Compare outputs from GPT-4o, Claude, DeepSeek Coder, Qwen
models = {
    "gpt-4o": "OpenAI",
    "claude-3-5-sonnet": "Anthropic",
    "deepseek-coder": "DeepSeek",
    "qwen2.5-coder": "Qwen (local via Ollama)"
}
```

Evaluation Metrics vs Outcomes

Traditional metrics (BLEU, ROUGE) don't capture real-world quality. Use LLM-as-Judge, human evaluation, and task-specific tests.

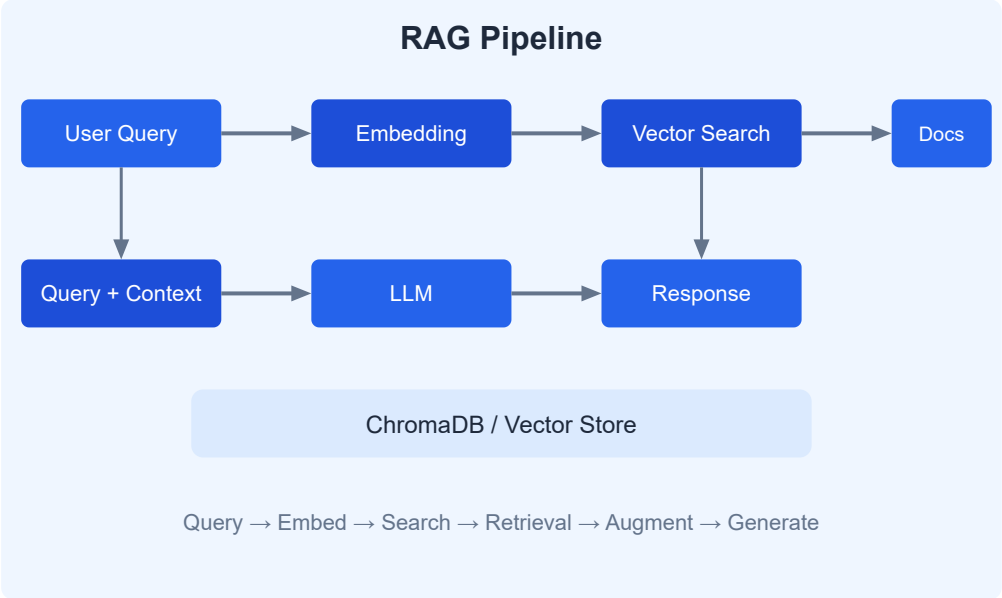
> 🏏 **Exercise:** Create a model comparison test: take 5 Python functions and ask 3 different models (GPT-4o, Claude, and a local model via Ollama) to translate them to Rust. Compare the outputs for correctness, efficiency, and readability. Build a Gradio UI to display results side by side.

← Back to Tutorials

Mastering RAG

Fundamentals

RAG grounds LLM responses in retrieved data, reducing hallucinations and improving accuracy. Key components: embedding model, vector store, and LLM.



LangChain & Vector Databases

```
pip install langchain chromadb sentence-transformers
```

Chunking and Indexing

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings

splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_text(document)

embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = Chroma.from_texts(chunks, embeddings)
```

Complete RAG Pipeline

```
from langchain.chains import RetrievalQA

qa = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vectorstore.as_retriever(search_kwargs={"k": 3})
)
result = qa.invoke("What is the capital of France?")
print(result)
```

RAG Evaluations

- **Faithfulness** — Does the answer match the retrieved context?
- **Relevance** — Is the retrieved context relevant to the query?
- **LLM-as-Judge** — Use an LLM to rate answer quality

Advanced RAG Techniques

- **Pre-processing** — Clean documents before chunking
- **Re-ranking** — Re-rank retrieved chunks with a cross-encoder
- **Query Expansion** — Generate multiple queries from one question
- **GraphRAG** — Build a knowledge graph from documents for better retrieval

✏️ **Exercise:** Build a complete RAG pipeline with conversation history. Use LangChain, ChromaDB, and a Gradio chat interface. Implement: (1) document upload and chunking with overlap, (2) vector embedding and indexing, (3) hybrid search (keyword + vector), (4) conversation-aware retrieval, and (5) an LLM-as-Judge evaluation step.

← Back to Tutorials

ML to DL to Fine-Tuning

Training & Datasets

Start with a capstone project using Amazon product data from HuggingFace datasets.

```
from datasets import load_dataset

dataset = load_dataset("amazon_polarity", split="train[:1000]")
print(dataset[0])
```

Baseline Models with Traditional ML

```
from sklearn.feature_extraction.text import TfidfVectorizer
from xgboost import XGBClassifier

vectorizer = TfidfVectorizer(max_features=5000)
X_train = vectorizer.fit_transform(train_texts)
model = XGBClassifier()
model.fit(X_train, train_labels)
```

Neural Networks with PyTorch

```
import torch
import torch.nn as nn

class SentimentClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.fc = nn.Linear(embed_dim, 2)

    def forward(self, x):
        x = self.embedding(x).mean(dim=1)
        return self.fc(x)
```

Fine-Tuning Frontier Models

Supervised Fine-Tuning (SFT) adapts a pre-trained model to your specific task. Use LoRA for efficient fine-tuning.

```
from transformers import AutoModelForCausalLM, TrainingArguments
from trl import SFTTrainer

model = AutoModelForCausalLM.from_pretrained("microsoft/Phi-3-mini-4k-instruc
trainer = SFTTrainer(
    model=model,
    train_dataset=dataset,
    args=TrainingArguments(output_dir="./phi3-finetuned", per_device_train_ba
)
trainer.train()
```

Monitoring & Handling Failures

- Track loss curves to detect overfitting
- Use gradient checkpointing for memory efficiency
- Save checkpoints every N steps
- Resume from checkpoints on failure

> ✏️ **Exercise:** Fine-tune a small language model (Phi-3 mini or Llama 3.2 1B) on a custom dataset of your choice. Start with an XGBoost baseline, then build a PyTorch classifier, then SFT a pre-trained model. Compare all three approaches on the same test set.

← Back to Tutorials

Fine-Tuned Open-Source Models

QLoRA Basics

QLoRA (Quantized Low-Rank Adaptation) makes fine-tuning large models accessible by combining 4-bit quantization with low-rank adapters.

```
pip install bitsandbytes peft trl datasets
```

Loading a Quantized Model

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model

quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16
)

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b-chat-hf",
    quantization_config=quant_config
)

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05
)

model = get_peft_model(model, lora_config)
```

Dataset Preparation

Optimize token length, format as chat templates, and split train/validation.

```
def format_example(example):
    return {
        "text": f"<|user|>\n{example['question']}\n<|assistant|>\n{example['a
    }

dataset = dataset.map(format_example)
```

Hyperparameter Configuration

Use Weights & Biases (W&B) for experiment tracking and HuggingFace TRL for the trainer.

```
from trl import SFTTrainer

trainer = SFTTrainer(
    model=model,
    train_dataset=dataset,
    args=TrainingArguments(
        output_dir="./lora-llama",
        per_device_train_batch_size=2,
        gradient_accumulation_steps=4,
        learning_rate=2e-4,
        logging_steps=10,
        save_steps=100,
        report_to="wandb"  # Track with W&B
    )
)
```

Monitoring Training

Watch for loss convergence, learning rate scheduling, and overfitting via validation loss.

Inference & Evaluation

Compare fine-tuned model outputs against the base model and frontier models on test prompts.

> ✏️ **Exercise:** Fine-tune Llama 3.2 1B or Phi-3 mini using QLoRA on a domain-specific dataset (e.g., medical Q&A, legal documents, code generation). Track training with W&B. Evaluate the fine-tuned model against the base model and a frontier model (GPT-4o) on 10 test prompts. Report metrics and qualitative comparisons.

← Back to Tutorials

Autonomous Multi-Agent Systems

Agentic AI & Deployment

Deploy agents to serverless cloud platforms like Modal for scalable, pay-per-use inference.

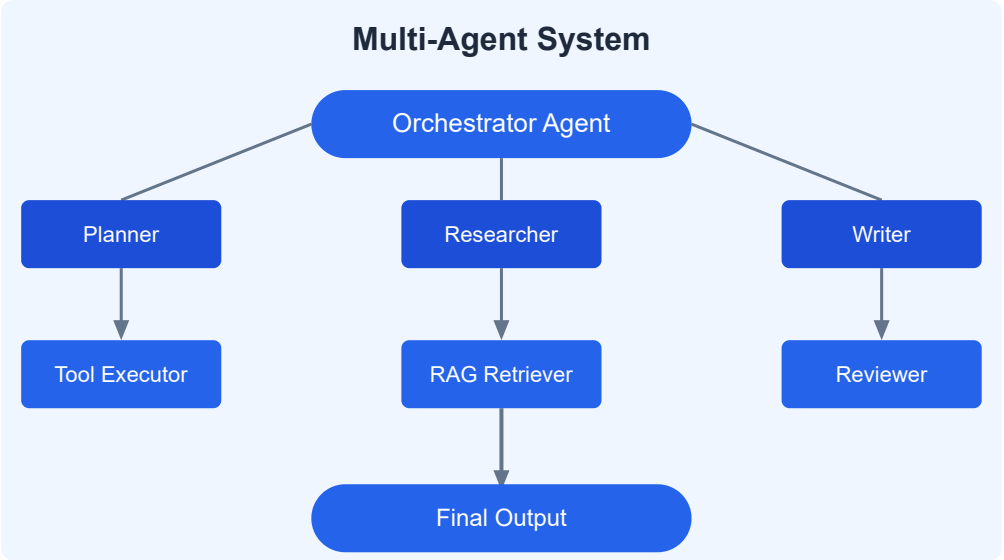
```
pip install modal
```

```
import modal

app = modal.App("agent-app")

@app.function(gpu="A100")
def run_agent(prompt: str):
    # Load model and generate
    return response

@app.local_entrypoint()
def main():
    print(run_agent.remote("Analyze this data"))
```

Advanced RAG with ChromaDB

Use ensemble retrieval with multiple embedding models for better coverage.

```
from langchain.retrievers import EnsembleRetriever

retriever = EnsembleRetriever(
    retrievers=[chroma_retriever, bm25_retriever],
    weights=[0.7, 0.3]
)
```



Structured Outputs with Pydantic

```
from pydantic import BaseModel
from typing import List

class Deal(BaseModel):
    title: str
    price: float
    discount: float
    url: str
    category: str

class DealScanResult(BaseModel):
    deals: List[Deal]
    total_savings: float
```

Planning Agents

Build agents that plan, execute, and evaluate — creating a Plan → Execute → Review loop.

```
class PlanningAgent:
    def plan(self, task):
        return self.llm(f"Create a step-by-step plan for: {task}")

    def execute(self, plan):
        for step in plan:
            result = self.execute_step(step)
            yield result

    def review(self, results):
        return self.llm(f"Evaluate these results: {results}")
```

Final Project: DealAgentFramework

Build a price-is-right deal scanner agent with:

- Web scraping MCP server for deal collection
- RAG agent for deal analysis and comparison
- Planning agent for deal monitoring schedule
- Gradio dashboard for visualization
- Modal deployment for 24/7 operation

> 🏏 **Exercise:** Build a DealAgentFramework: (1) create an MCP server that scrapes deal websites, (2) a RAG agent that stores and retrieves deals, (3) a planning agent that monitors prices and alerts on drops, (4) use Pydantic for structured deal output, and (5) deploy to Modal with a Gradio frontend.

← Back to Tutorials

9. Practice & Resources

Quick Reference Guide

Task	Library / Tool	Command
Run local LLM	Ollama	`ollama pull llama3.2`
Chat completions	openai / litellm	`litellm --model gpt-4o`
Vector search	ChromaDB	`Chroma.from_texts(chunks, embeddings)`
RAG pipeline	LangChain	`RetrievalQA.from_chain_type(...)`
Fine-tuning	TRL + PEFT	`SFTTrainer(model, dataset)`
QLoRA	bitsandbytes	`load_in_4bit=True`
UI prototyping	Gradio	`gr.ChatInterface(...).launch()`
Serverless deploy	Modal	`modal deploy app.py`

Useful Resources

- Ollama — Local LLM runner
- HuggingFace — Models, datasets, Spaces
- LangChain — RAG and agent framework
- Gradio — ML demo interfaces
- Modal — Serverless GPU compute
- TRL (Transformer Reinforcement Learning)

8-Week Study Plan

Week	Topic	Activities
1	LLM Product	Ollama, LLM types, streaming, brochure generator
2	Multi-Modal Chatbot	Gradio, tool calling, DALL-E, TTS, SQLite
3	HuggingFace	Pipelines, tokenizers, quantization, Whisper
4	LLM Showdown	Model comparison, benchmarks, code gen
5	RAG	LangChain, ChromaDB, evaluations, advanced RAG
6	ML/DL/Fine-Tuning	XGBoost, PyTorch, SFT, monitoring
7	Open-Source Fine-Tuning	QLoRA, adapters, W&B, evaluation
8	Multi-Agent Systems	Planning agents, Modal, deal scanner capstone

Certificate Path

- **AI-900:** Azure AI Fundamentals
- **AI-102:** Azure AI Engineer Associate
- **HuggingFace Course** — Free NLP and diffusion model training

> 🚀 **Final Capstone:** Combine everything into a production-ready system: (1) fine-tune a small model on your domain data using QLoRA, (2) deploy it with Modal, (3) build a RAG pipeline around it with ChromaDB, (4) add a Gradio chat interface, (5) implement multi-agent orchestration with planning and review, and (6) monitor with W&B logging.

← Back to Tutorials

10. References

Glossary

Term	Definition
LLM	Large Language Model — neural network trained on vast text data
Transformer	Neural architecture using self-attention
Tokenization	Converting text to subword token IDs
Embedding	Dense vector representation of text
RAG	Retrieval-Augmented Generation — ground output in retrieved data
SFT	Supervised Fine-Tuning — training on labeled examples
QLoRA	Quantized Low-Rank Adaptation — efficient fine-tuning
Quantization	Reducing model precision (32-bit to 4-bit)
Agent	AI system that uses tools, memory, and planning autonomously

Cheatsheet

```
# Installation Cheatsheet
pip install ollama                    # Local LLMs
pip install langchain chromadb        # RAG
pip install transformers torch        # HuggingFace
pip install bitsandbytes peft trl     # Fine-tuning
pip install gradio                    # UI
pip install modal                     # Serverless

# Quick inference (HuggingFace)
from transformers import pipeline
pipe = pipeline("text-generation", model="gpt2")
pipe("Once upon a time")

# Quick RAG (LangChain + Chroma)
from langchain.chains import RetrievalQA
qa = RetrievalQA.from_chain_type(llm=llm, retriever=vectorstore.as_retriever(

# Quick fine-tuning (TRL)
from trl import SFTTrainer
trainer = SFTTrainer(model=model, train_dataset=dataset)
trainer.train()
```

Model Comparison

Model	Size	Best For	Access
GPT-4o	Frontier	General, multimodal, coding	API
Claude 3.5	Frontier	Analysis, writing, safety	API
Llama 3.2	1B-90B	Open-source, local deploy	Ollama / HF
Phi-3 / Phi-4	3.8B-14B	Efficient, edge devices	HF / Azure
DeepSeek V3 / R1	67B-671B	Reasoning, code, math	API / Ollama
Qwen 2.5	0.5B-72B	Multilingual, code	Ollama / HF

📘 **Congratulations!** You've completed the AI Engineer Core curriculum. You can now build LLM products, multi-modal chatbots, RAG pipelines, fine-tune models, and deploy multi-agent systems to production.